Отчёт по лабораторной работе 2

Потапов С. А. НКАбд-05-25

Потапов Савелий Александрович

Содержание

1 Цель работы 5

2 Задание 6

3 Выполнение лабораторной работы 7

4 Выводы 14

5 Ответы на контрольные вопросы 15

Список иллюстраций

Список таблиц

1 Цель работы

- Изучить идеологию и применение средств контроля версий.
- Освоить умения по работе с git.

2 Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ SSH.
- Создать ключ PGP.
- Настроить подписи git.
- Зарегистрироваться на Github.
- Создать локальный каталог для выполнения заданий по предмету.

3 Выполнение лабораторной работы

```
(base) sapotapov@sapotapov:~$ sudo dnf install git
[sudo] пароль для sapotapov:
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет "git-2.49.0-1.fc42.x86_64" уже установлен.

Нечего делать.
(base) sapotapov@sapotapov:~$
```

Я установил git.

```
(base) sapotapov@sapotapov:~$ sudo dnf install gh
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет                              Арх.      Версия                          Репозиторий          Размер
Установка:
 gh                                x86_64    2.79.0-1.fc42                   updates            54.3 MiB

Сводка транзакции:
 Установка:         1 пакета

Общий размер входящих пакетов составляет 14 MiB. Необходимо загрузить 14 MiB.
После этой операции будут использоваться дополнительные 54 MiB (установка 54 MiB, удаление 0 B).
Is this ok [y/N]: y
[1/1] gh-0:2.79.0-1.fc42.x86_64                                  100% |  13.1 MiB/s |  14.2 MiB |  00m01s
--------------------------------------------------------------------------------------------------------
[1/1] Total                                                      100% |  10.2 MiB/s |  14.2 MiB |  00m01s
Выполнение транзакции
[1/3] Проверить файлы пакета                 100% |  10.0   B/s |   1.0   B |  00m00s00s
[2/3] Подготовить транзакцию                 100% |   1.0   B/s |   1.0   B |  00m01s00s
[3/3] Установка gh-0:2.79.0-1.fc42.x86_64               100% |  44.0 MiB/s |  54.4 MiB |  00m01s
Завершено!
(base) sapotapov@sapotapov:~$
```

Я установил gh.

```
(base) sapotapov@sapotapov:~$ git config --global user.name "Saveliy Potapov"
(base) sapotapov@sapotapov:~$ git config --global user.email "1032253503@rudn.ru"
(base) sapotapov@sapotapov:~$ git config --global core.quotepath false
```

Была проведена базовая настройка: Я задал своё имя, email, настроил utf-8 в выводе сообщений git.

```
(base) sapotapov@sapotapov:~$ git config --global init.defaultBranch master
(base) sapotapov@sapotapov:~$ git config --global core.autocrlf input
(base) sapotapov@sapotapov:~$ gi config --global core.safecrlf warn
bash: gi: команда не найдена...
(base) sapotapov@sapotapov:~$ git config --global core.safecrlf warn
```

Я настроил верификацию и подписание коммитов git, задал имя начальной ветки, параметры autocrlf и safecrlf.

```
(base) sapotapov@sapotapov:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/sapotapov/.ssh/id_rsa): rsa4096test
Enter passphrase for "rsa4096test" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in rsa4096test
Your public key has been saved in rsa4096test.pub
The key fingerprint is:
SHA256:C3uc3ONrfLaNdrjrXTf5KRfF1VOBlTHtknWPgSfASlY sapotapov@sapotapov
The key's randomart image is:
+---[RSA 4096]----+
|         oE. .o=O|
|        o . o.ooB|
|       o .   o B=|
|        .     + =|
|       . S      ..|
|        = +     ..|
|       . *.o  . o+|
|        . .o.==.o=|
|         .o**==..|
+----[SHA256]-----+
```

Я создал ключ по алгоритму rsa с размером 4096 бит.

```
(base) sapotapov@sapotapov:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sapotapov/.ssh/id_ed25519): ed25519test
Enter passphrase for "ed25519test" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ed25519test
Your public key has been saved in ed25519test.pub
The key fingerprint is:
SHA256:6rF3RpXuKVi6hFb+WRutm1i6ox6R3sHxrV8xKaCIp2M sapotapov@sapotapov
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|                 |
|          .. .   |
|      . .o.o+.  .|
|     . oS.oo...+ |
|      o* oo..o. o|
|     E= ==..=.. .|
|    .o.++o=*o* . |
|      oo+=*+=..  |
+----[SHA256]-----+
```

Я создал ключ по алгоритму ed25519 с размером 4096 бит.

```
gpg: создан каталог '/home/sapotapov/.gnupg'
Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Saveliy Potapov Aleksandrovich
Адрес электронной почты: 1032253503@rudn.ru
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "Saveliy Potapov Aleksandrovich <1032253503@rudn.ru>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? O
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: /home/sapotapov/.gnupg/trustdb.gpg: создана таблица доверия
gpg: создан каталог '/home/sapotapov/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/home/sapotapov/.gnupg/openpgp-revocs.d/B6B32FD9F3E0C79A97C8EB31DE15018
694FFBF8D.rev'.
открытый и секретный ключи созданы и подписаны.

pub   rsa4096 2026-03-04 [SC]
      B6B32FD9F3E0C79A97C8EB31DE15018694FFBF8D
uid                      Saveliy Potapov Aleksandrovich <1032253503@rudn.ru>
sub   rsa4096 2026-03-04 [E]
```

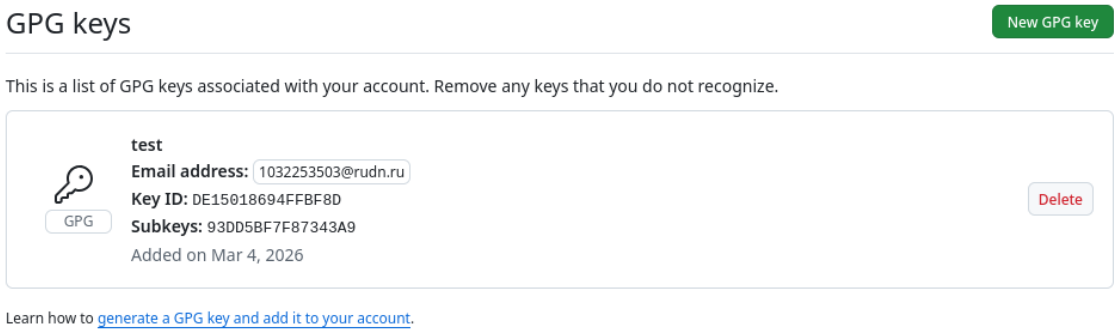
Я сгенерировал ключ, после чего создал аккаунт на github.com

```
(base) sapotapov@sapotapov:~$ gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: глубина: 0  достоверных:   1  подписанных:   0  доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
---------
sec   rsa4096/DE15018694FFBF8D 2026-03-04 [SC]
      B6B32FD9F3E0C79A97C8EB31DE15018694FFBF8D
uid               [  абсолютно ] Saveliy Potapov Aleksandrovich <1032253503@rudn.ru>
ssb   rsa4096/93DD5BF7F87343A9 2026-03-04 [E]
```

Я вывел список ключей и скопировал отпечаток приватного ключа.

```
(base) sapotapov@sapotapov:~$ gpg --armor --export B6B32FD9F3E0C79A97C8EB31DE15018694FFBF8D | xclip -sel
clip
```

Я скопировал сгенерированный ключ в буфер обмена.



В настройках Github я вставил свой GPG ключ.

```
(base) sapotapov@sapotapov:~$ git config --global user.signingkey B6B32FD9F3E0C79A97C8EB31DE15018694FFBF8
D
(base) sapotapov@sapotapov:~$ git config --global commit.gpgsign true
(base) sapotapov@sapotapov:~$ git config --global gpg.program $(which gpg2)
```

Я указал Git применять мой адрес электронной почты при подписи коммитов.

```
(base) sapotapov@sapotapov:~$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 1FD0-ADF1
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as sapotapovrudn25
```

Я авторизовался в gh.

```
(base) sapotapov@sapotapov:~$ mkdir -p ~/work/study/2025-2026/"Операционные системы"
(base) sapotapov@sapotapov:~$ cd  ~/work/study/2025-2026/"Операционные системы"
(base) sapotapov@sapotapov:~/work/study/2025-2026/Операционные системы$ gh repo create study_2025-2026_os
-intro --template=yamadharma/course-directory-student-template --public
✓ Created repository sapotapovrudn25/study_2025-2026_os-intro on github.com
  https://github.com/sapotapovrudn25/study_2025-2026_os-intro
(base) sapotapov@sapotapov:~/work/study/2025-2026/Операционные системы$ git clone --recursive git@github.
com:<owner>/study_2025-2026_os-intro.git os-intro
bash: owner: Нет такого файла или каталога
(base) sapotapov@sapotapov:~/work/study/2025-2026/Операционные системы$ git clone --recursive git@github.
com:<owner>/study_2022-2023_os-intro.git os-intro
bash: owner: Нет такого файла или каталога
(base) sapotapov@sapotapov:~/work/study/2025-2026/Операционные системы$ git clone --recursive git@github.
com:sapotapovrudn25/study_2025-2026_os-intro.git os-intro
Клонирование в «os-intro»...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 41 (delta 1), reused 28 (delta 1), pack-reused 0 (from 0)
Получение объектов: 100% (41/41), 25.54 КиБ | 12.77 МиБ/с, готово.
Определение изменений: 100% (1/1), готово.
Подмодуль «template/presentation» (https://github.com/yamadharma/academic-presentation-markdown-template.
git) зарегистрирован по пути «template/presentation»
Подмодуль «template/report» (https://github.com/yamadharma/academic-laboratory-report-template.git) зарег
истрирован по пути «template/report»
Клонирование в «/home/sapotapov/work/study/2025-2026/Операционные системы/os-intro/template/presentation»
...
remote: Enumerating objects: 239, done.
remote: Counting objects: 100% (239/239), done.
remote: Compressing objects: 100% (168/168), done.
remote: Total 239 (delta 96), reused 202 (delta 59), pack-reused 0 (from 0)
Получение объектов: 100% (239/239), 2.67 МиБ | 8.17 МиБ/с, готово.
Определение изменений: 100% (96/96), готово.
Клонирование в «/home/sapotapov/work/study/2025-2026/Операционные системы/os-intro/template/report»...
remote: Enumerating objects: 287, done.
remote: Counting objects: 100% (287/287), done.
remote: Compressing objects: 100% (195/195), done.
remote: Total 287 (delta 125), reused 234 (delta 72), pack-reused 0 (from 0)
Получение объектов: 100% (287/287), 785.00 КиБ | 3.85 МиБ/с, готово.
Определение изменений: 100% (125/125), готово.
Submodule path 'template/presentation': checked out '60a6091ebedbb01a8d98f2c5fae0f98dc63ad031'
Submodule path 'template/report': checked out '708b8c51acd94d4b78e7634198f221e5decb6355'
```

Я создал директорию для репозитория курса и скопировал сам репозиторий.

```
(base) sapotapov@sapotapov:~/work/study/2025-2026/Операционные системы/os-intro$ git add .
(base) sapotapov@sapotapov:~/work/study/2025-2026/Операционные системы/os-intro$ git commit -am 'feat(mai
n): make course structure'
[master 72a3e86] feat(main): make course structure
 2 files changed, 1 insertion(+), 14 deletions(-)
 delete mode 100644 package.json
```

```
(base) sapotapov@sapotapov:~/work/study/2025-2026/Операционные системы/os-intro$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 12 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 950 байтов | 950.00 КиБ/с, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:sapotapovrudn25/study_2025-2026_os-intro.git
   6405bba..72a3e86  master -> master
```

Я отправил файлы из нового каталога на сервер, зафиксировав коммит.

4 Выводы

В процессе проделанной работы я научился работать с системой контроля версий Git через командную строку моего устройства. Эти навыки пригодятся мне в дальнейшем обучении и на работе.

5 Ответы на контрольные вопросы

1) Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются? Система контроля версий - хранилище, содержащее разные версии одного проекта. Она применяется при одновременной работе нескольких человек над одним проектом и позволяет добавлять новые версии, сравнивать версии, совмещать изменения из разных версий, откатывать проект к более ранней версии.

2) Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия. Хранилище - Виртуальное пространство, в котором в котором хранятся различные версии проекта. Commit - версия проекта с указанием изменений. История - Список версий проекта, в котором можно отследить все коммиты по дате. Рабочая копия - локально сохранённая версия проекта, которую изменяют перед тем как загрузить на сервер.

3) Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида. Централизованная модель VCS предполагает наличие единого репозитория для хранения файлов, из которого участники проекта получают нужные им версии файлов. В децентрализованной модели необязательно наличие главного репозитория. Примеры классических VCS: CVS, Subversion. Примеры децентрализованных: Git, Bazaar, Mercurial.

4) Опишите действия с VCS при единоличной работе с хранилищем. Получаем изменения из центрального репозитория. Вносим изменения на локальном дереве, указываем системе, какие файлы не добавлять в центральный репозиторий. Затем, сохраняем изменения локально, фиксируем их на сервере и размещаем файлы в центральном репозитории.

5) Опишите порядок работы с общим хранилищем VCS. Получаем изменения из центрального репозитория. Вносим изменения на локальном дереве, указываем системе, какие файлы не добавлять в центральный репозиторий. Затем, сохраняем изменения локально, фиксируем их на сервере и размещаем файлы в центральном репозитории.

6) Каковы основные задачи, решаемые инструментальным средством git? Git является системой контроля версий, а следовательно, главная задача, решаемая с его помощью - отслеживание различных версий проекта и загрузка их на сервер.

7) Назовите и дайте краткую характеристику командам git. Создание основного дерева репозитория: git init Получение обновлений (изменений) текущего дерева из центрального репозитория: git pull Отправка всех произведённых изменений локального дерева в центральный репозиторий: git push Просмотр списка изменённых файлов в текущей директории: git status Просмотр текущих изменений: git diff Сохранение текущих изменений: Добавить все изменённые и/или созданные файлы и/или каталоги: git add . Добавить конкретные изменённые и/или созданные файлы и/или каталоги: git add имена_файлов Удалить файл и/или каталог из индекса репозитория (при этом файл и/или каталог остаётся в локальной директории): git rm имена_файлов Сохранение добавленных изменений: Сохранить все добавленные изменения и все изменённые файлы: git commit -am „Описание коммита“ Сохранить добавленные изменения с

внесением комментария через встроенный редактор: git commit Создание новой ветки, базирующейся на текущей: git checkout -b имя_ветки Переключение на некоторую ветку(при переключении на ветку, которой ещё нет в локальном репозитории, она будет создана и связана с удалённой): git checkout имя_ветки Отправка изменений конкретной ветки в центральный репозиторий: git push origin имя_ветки Слияние ветки с текущим деревом: git merge –no-ff имя_ветки Удаление ветки: Удаление локальной уже слитой с основным деревом ветки: git branch -d имя_ветки Принудительное удаление локальной ветки: git branch -D имя_ветки Удаление ветки с центрального репозитория: git push origin :имя_ветки

8) Приведите примеры использования при работе с локальным и удалённым репозиториями. Локальный репозиторий - локально сохранённая версия репозитория, в которую мы вносим изменения перед тем как загрузить их на удалённый репозиторий. Удалённый репозиторий же используется для получения версии проекта и принимает загруженные изменения.

9) Что такое и зачем могут быть нужны ветви (branches)? Ветви используются, чтобы изолировать изменения проекта без риска нарушить работу главной версии. После завершения работы с ветвью, её можно объединить («merge») с главной ветвью репозитория.

10) Как и зачем можно игнорировать некоторые файлы при commit? Во время работы над проектом могут создаваться файлы, которые не требуется добавлять в последствии в репозиторий. Например, временные файлы, создаваемые редакторами, или объектные файлы, создаваемые компиляторами. Можно прописать шаблоны игнорируемых при добавлении в репозиторий типов файлов в файл .gitignore с помощью сервисов.